

Important Code Reference: build/installer.nsh

A focused code/control-file explanation covering purpose, owner layer, APIs, variables, implementation signals, source excerpts, and debugging rules.

Item	Explanation
File	build/installer.nsh
Category	NSIS installer script
Folder role	Packaging and installer customization. Used while creating the Windows app installer.
Size	29,416 bytes
Extension	.nsh
MIME type	unknown
Text lines	570

Why this file exists

- NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.

How it is used

Packaging and installer customization. Used while creating the Windows app installer.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Installer at line 6	Windows setup, update, shortcut, or installation behavior. Evidence: <code>!ifndef BUILD_UNINSTALLER</code>
Installer at line 8	Windows setup, update, shortcut, or installation behavior. Evidence: <code>Var OMB_ShortcutCheckbox</code>
Installer at line 10	Windows setup, update, shortcut, or installation behavior. Evidence: <code>Var OMB_CreateDesktopShortcutState</code>
Installer at line 24	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${NSD_CreateLabel} 0 0 100% 12u "Tools and shortcuts"</code>
Git command at line 27	Repository state, publishing, branch checks, or release automation. Evidence: <code>\${NSD_CreateLabel} 0 18u 100% 36u "The desktop app includes its own runtime. Users do not need to install Node.js or npm.\$\r\$\nGit for Windows with Git Credential Manager is only needed when publishing to GitHub."</code>
Installer at line 30	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${NSD_CreateCheckbox} 0 62u 100% 12u "Create a desktop shortcut"</code>
Installer at line 31	Windows setup, update, shortcut, or installation behavior. Evidence: <code>Pop \$OMB_ShortcutCheckbox</code>
Installer at line 32	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${If} \$OMB_CreateDesktopShortcutState == ""</code>
Installer at line 33	Windows setup, update, shortcut, or installation behavior. Evidence: <code>StrCpy \$OMB_CreateDesktopShortcutState \${BST_CHECKED}</code>
Installer at line 35	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${If} \$OMB_CreateDesktopShortcutState == \${BST_CHECKED}</code>
Installer at line 36	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${NSD_Check} \$OMB_ShortcutCheckbox</code>
Git command at line 39	Repository state, publishing, branch checks, or release automation. Evidence: <code>\${NSD_CreateCheckbox} 0 82u 100% 24u "Check publishing tools and install Git for Windows if Git/Git Credential Manager is missing"</code>
Git command at line 48	Repository state, publishing, branch checks, or release automation. Evidence: <code>\${NSD_CreateLabel} 0 114u 100% 38u "Existing shared tools are not uninstalled silently. If a compatible tool is already available, setup skips it. If Git is missing or unusable, setup will try to install Git for Windows"</code>

Item	Explanation
Installer at line 55	Windows setup, update, shortcut, or installation behavior. Evidence: \${NSD_GetState} \$OMB_ShortcutCheckbox \$OMB_CreateDesktopShortcutState
Git command at line 66	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$PROGRAMFILES64\Git\cmd\git.exe" 0 +2
Git command at line 68	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$PROGRAMFILES\Git\cmd\git.exe" 0 +2
Git command at line 70	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$PROGRAMFILES32\Git\cmd\git.exe" 0 +2
Git command at line 72	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$LOCALAPPDATA\Programs\Git\cmd\git.exe" 0 +2
Git command at line 76	Repository state, publishing, branch checks, or release automation. Evidence: nsExec::ExecToStack 'cmd /c git --version'
Security or auth at line 86	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Function OMBCheckGitCredentialManagerAvailable
Git command at line 87	Repository state, publishing, branch checks, or release automation. Evidence: nsExec::ExecToStack 'cmd /c git credential-manager --version'
Security or auth at line 100	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Call OMBCheckGitCredentialManagerAvailable
Git command at line 105	Repository state, publishing, branch checks, or release automation. Evidence: DetailPrint "Git for Windows and Git Credential Manager were found. Skipping publishing tool installation."
Git command at line 114	Repository state, publishing, branch checks, or release automation. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows was not found and Windows Package Manager is unavailable. Open Publishing target > Install Git inside the app, or install Git
Git command at line 118	Repository state, publishing, branch checks, or release automation. Evidence: DetailPrint "Installing or repairing Git for Windows. This can take several minutes."
Git command at line 119	Repository state, publishing, branch checks, or release automation. Evidence: ExecWait 'winget install --id Git.Git -e --source winget --accept-source-agreements --accept-package-agreements' \$0
Git command at line 121	Repository state, publishing, branch checks, or release automation. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows setup did not complete. Open Publishing target > Install Git inside the app when you are ready to publish."
Git command at line 123	Repository state, publishing, branch checks, or release automation. Evidence: DetailPrint "Git for Windows setup completed."
Installer at line 149	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder \$OMB_ExistingInstallVersion is already installed.\$r\$n\$r\$nThis installer is also version \${VERSION}, so setup will stop without changing the app."
Installer at line 157	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder \$OMB_ExistingInstallVersion is already installed.\$r\$n\$r\$nThis installer is older (\${VERSION}), so setup will stop without changing the app."
Installer at line 161	Windows setup, update, shortcut, or installation behavior. Evidence: DetailPrint "OMB Portfolio Builder \$OMB_ExistingInstallVersion is newer than installer \${VERSION}. Setup stopped."
Installer at line 168	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$PROFILE\OMB\builder\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing this releas
Installer at line 172	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$DOCUMENTS\OMB\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing this release. Thi
Installer at line 176	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$PROFILE\OneDrive\Documents\OMB\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing
Installer at line 180	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$PROFILE\Documents\OMB\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing this rele

Item	Explanation
Installer at line 184	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development copy:~\Documents\omb-portfolio-builder~\Remove, uninstall, or rename that copy before installing this release."
Installer at line 188	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development copy:~\PROFILE\OneDrive\Documents\omb-portfolio-builder~\Remove, uninstall, or rename that copy before installi
File read at line 208	Reads local builder state, project files, website assets, or configuration. Evidence: FileWrite \$0 " try { \$\$\$packageText = Get-Content -LiteralPath (Join-Path \$\$\$Directory 'package.json') -Raw -ErrorAction Stop } catch { return \$\$\$false }~\n"
File read at line 215	Reads local builder state, project files, website assets, or configuration. Evidence: FileWrite \$0 " try { \$\$\$packageText = Get-Content -LiteralPath (Join-Path \$\$\$Directory 'package.json') -Raw -ErrorAction Stop } catch { return \$\$\$false }~\n"
Installer at line 220	Windows setup, update, shortcut, or installation behavior. Evidence: FileWrite \$0 " \$\$\$managedRoot = Join-Path \$\$\$env:LOCALAPPDATA 'OMB Portfolio Builder'~\n"
Git command at line 227	Repository state, publishing, branch checks, or release automation. Evidence: FileWrite \$0 " if (\$\$name -in @('node_modules','git','dist','.vs','\$\$Recycle.Bin','System Volume Information','WinSxS')) { return \$\$\$true }~\n"
Installer at line 251	Windows setup, update, shortcut, or installation behavior. Evidence: FileWrite \$0 "\$\$registryKeys = @('HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall*', 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Uninstall*')~\n"
Installer at line 261	Windows setup, update, shortcut, or installation behavior. Evidence: FileWrite \$0 "foreach (\$\$common in @(\$\$env:ProgramFiles, \$\$\$programFilesX86, (Join-Path \$\$\$env:LOCALAPPDATA 'Programs'), \$\$\$env:USERPROFILE, (Join-Path \$\$\$env:PUBLIC 'Desktop'))){ if (\$\$common -and -not \$\$\$roots.Contains(\$\$c
Installer at line 282	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found another OMB Portfolio Builder copy on this machine.~\n~\n~\n~\nTo prevent duplicate installations and stale builder features, remove, uninstall, or rename the exis
Installer at line 285	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup could not complete the duplicate-installation scan. To avoid duplicate builder copies, setup will stop. Run the installer again as Administrator, or remove old OMB Portfolio Builder co
Installer at line 298	Windows setup, update, shortcut, or installation behavior. Evidence: Function OMBDisableBuiltInOldUninstallerForInPlaceUpdate
Installer at line 299	Windows setup, update, shortcut, or installation behavior. Evidence: DetailPrint "Preparing in-place update. The old uninstaller handoff will be skipped."
Installer at line 302	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 303	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 304	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 305	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 308	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 309	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 310	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 311	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 315	Windows setup, update, shortcut, or installation behavior. Evidence: Function OMBUninstallExistingInstallIfPresent
Installer at line 326	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" DisplayVersion
Installer at line 327	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 329	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" UninstallString

Item	Explanation
Installer at line 332	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" InstallLocation
Installer at line 337	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" DisplayVersion
Installer at line 338	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 340	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" UninstallString
Installer at line 343	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" InstallLocation
Installer at line 350	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" DisplayVersion
Installer at line 351	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 353	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" UninstallString
Installer at line 356	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" InstallLocation
Installer at line 360	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" DisplayVersion
Installer at line 361	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 363	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" UninstallString
Installer at line 366	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" InstallLocation
Installer at line 373	Windows setup, update, shortcut, or installation behavior. Evidence: IfFileExists "\$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
Installer at line 374	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R7 "\$LOCALAPPDATA\Programs\OMB Portfolio Builder"
Installer at line 375	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R5 ""\$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
Installer at line 378	Windows setup, update, shortcut, or installation behavior. Evidence: IfFileExists "\$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
Installer at line 380	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R5 ""\$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
Installer at line 383	Windows setup, update, shortcut, or installation behavior. Evidence: IfFileExists "\$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
Installer at line 385	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R5 ""\$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
Installer at line 440	Windows setup, update, shortcut, or installation behavior. Evidence: DetailPrint "Migrating legacy OMB application install to the AppData per-user install location."

Function-by-function detail

OMBToolsPageCreate (build/installer.nsh, lines 17-52)

OMBToolsPageCreate named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 17-52 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
17: Function OMBToolsPageCreate
18:   nsDialogs::Create 1018
19:   Pop $OMB_ToolsDialog
20:   ${If} $OMB_ToolsDialog == error
21:     Abort
22:   ${EndIf}
23:
24:   ${NSD_CreateLabel} 0 0 100% 12u "Tools and shortcuts"
25:   Pop $0
26:
27:   ${NSD_CreateLabel} 0 18u 100% 36u "The desktop app includes its own runtime. Users do not need to
install Node.js or pnpm.$r$\nGit for Windows with Git Credential Manager is only needed when publishing to
GitHub."
28:   Pop $0
29:
30:   ${NSD_CreateCheckbox} 0 62u 100% 12u "Create a desktop shortcut"
31:   Pop $OMB_ShortcutCheckbox
32:   ${If} $OMB_CreateDesktopShortcutState == ""
33:     StrCpy $OMB_CreateDesktopShortcutState ${BST_CHECKED}
34:   ${EndIf}
35:   ${If} $OMB_CreateDesktopShortcutState == ${BST_CHECKED}
36:     ${NSD_Check} $OMB_ShortcutCheckbox
37:   ${EndIf}
38:
39:   ${NSD_CreateCheckbox} 0 82u 100% 24u "Check publishing tools and install Git for Windows if Git/Git
Credential Manager is missing"
40:   Pop $OMB_PublishingToolsCheckbox
41:   ${If} $OMB_InstallPublishingToolsState == ""
42:     StrCpy $OMB_InstallPublishingToolsState ${BST_CHECKED}
43:   ${EndIf}
44:   ${If} $OMB_InstallPublishingToolsState == ${BST_CHECKED}
45:     ${NSD_Check} $OMB_PublishingToolsCheckbox
46:   ${EndIf}
47:
48:   ${NSD_CreateLabel} 0 114u 100% 38u "Existing shared tools are not uninstalled silently. If a compatible
tool is already available, setup skips it. If Git is missing or unusable, setup will try to install Git for
Windows side-by-side using Windows Package Manager."
49:   Pop $0
50:
51:   nsDialogs::Show
52: FunctionEnd
```

OMBToolsPageLeave (build/installer.nsh, lines 54-62)

OMBToolsPageLeave named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 54-62 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
54: Function OMBToolsPageLeave
55:   ${NSD_GetState} $OMB_ShortcutCheckbox $OMB_CreateDesktopShortcutState
56:   ${NSD_GetState} $OMB_PublishingToolsCheckbox $OMB_InstallPublishingToolsState
57:   ${If} $OMB_IsUpdateInstall == "1"
58:     ${AndIf} $INSTDIR != $OMB_ExistingInstallLocation
59:       MessageBox MB_OK|MB_ICONSTOP "OMB Portfolio Builder is already installed on this
machine.$r$\n$r$\nUpdates must use the current installed
location:$r$\n$OMB_ExistingInstallLocation$r$\n$r$\nSetup will not create another installed copy in a
different directory."
60:     Abort
61:   ${EndIf}
62: FunctionEnd
```

OMBCheckGitAvailable (build/installer.nsh, lines 64-84)

OMBCheckGitAvailable part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 64-84 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
64: Function OMBCheckGitAvailable
65:   StrCpy $R9 "0"
66:   IfFileExists "$PROGRAMFILES64\Git\cmd\git.exe" 0 +2
67:   StrCpy $R9 "1"
68:   IfFileExists "$PROGRAMFILES\Git\cmd\git.exe" 0 +2
69:   StrCpy $R9 "1"
70:   IfFileExists "$PROGRAMFILES32\Git\cmd\git.exe" 0 +2
71:   StrCpy $R9 "1"
72:   IfFileExists "$LOCALAPPDATA\Programs\Git\cmd\git.exe" 0 +2
73:   StrCpy $R9 "1"
74:
75:   ${If} $R9 == "0"
76:     nsExec::ExecToStack 'cmd /c git --version'
77:     Pop $0
78:     Pop $1
79:     ${If} $0 == 0
80:       StrCpy $R9 "1"
81:     ${EndIf}
82:   ${EndIf}
83:   Push $R9
84: FunctionEnd
```

OMBCheckGitCredentialManagerAvailable (build/installer.nsh, lines 86-95)

OMBCheckGitCredentialManagerAvailable part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 86-95 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
86: Function OMBCheckGitCredentialManagerAvailable
87:   nsExec::ExecToStack 'cmd /c git credential-manager --version'
88:   Pop $0
89:   Pop $1
90:   ${If} $0 == 0
91:     Push "1"
92:   ${Else}
93:     Push "0"
94:   ${EndIf}
95: FunctionEnd
```

OMBInstallGitIfNeeded (build/installer.nsh, lines 97-125)

OMBInstallGitIfNeeded part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 97-125 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
97: Function OMBInstallGitIfNeeded
98:   Call OMBCheckGitAvailable
99:   Pop $R9
100:   Call OMBCheckGitCredentialManagerAvailable
101:   Pop $R8
102:
103:   ${If} $R9 == "1"
```

```

104:    ${AndIf} $R8 == "1"
105:    DetailPrint "Git for Windows and Git Credential Manager were found. Skipping publishing tool
installation."
106:    Return
107:    ${EndIf}
108:
109:    DetailPrint "Publishing tools are missing or incomplete. Checking Windows Package Manager."
110:    nsExec::ExecToStack 'where.exe winget'
111:    Pop $0
112:    Pop $1
113:    ${If} $0 != 0
114:        MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows was not
found and Windows Package Manager is unavailable. Open Publishing target > Install Git inside the app, or
install Git for Windows with Git Credential Manager from https://git-scm.com/download/win."
115:        Return
116:    ${EndIf}
117:
118:    DetailPrint "Installing or repairing Git for Windows. This can take several minutes."
119:    ExecWait 'winget install --id Git.Git -e --source winget --accept-source-agreements --accept-package-
agreements' $0
120:    ${If} $0 != 0
121:        MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows setup
did not complete. Open Publishing target > Install Git inside the app when you are ready to publish."
122:    ${Else}
123:        DetailPrint "Git for Windows setup completed."
124:    ${EndIf}
125: FunctionEnd

```

OMBReadExistingAppVersion (build/installer.nsh, lines 127-139)

OMBReadExistingAppVersion named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 127-139 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Write. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

127: Function OMBReadExistingAppVersion
128:   StrCpy $OMB_ExistingInstallVersion ""
129:   IfFileExists "$R7\OMB Portfolio Builder.exe" omb_read_existing_app_version 0
130:   Return
131:
132:   omb_read_existing_app_version:
133:   nsExec::ExecToStack 'powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "$$p = '$R7\OMB
Portfolio Builder.exe'; try { $$v = (Get-Item -LiteralPath $$p).VersionInfo.FileVersion; if (-not $$v) { $$v =
(Get-Item -LiteralPath $$p).VersionInfo.ProductVersion }; [Console]::Write($$v) } catch { exit 1 }'"
134:   Pop $R4
135:   Pop $R3
136:   ${If} $R4 == 0
137:       StrCpy $OMB_ExistingInstallVersion $R3
138:   ${EndIf}
139: FunctionEnd

```

OMBStopIfExistingInstallIsCurrent (build/installer.nsh, lines 141-164)

OMBStopIfExistingInstallIsCurrent named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 141-164 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references older. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

141: Function OMBStopIfExistingInstallIsCurrent
142:   ${If} $OMB_ExistingInstallVersion == ""
143:       Return
144:   ${EndIf}
145:

```

```

146:   ${VersionCompare} "$OMB_ExistingInstallVersion" "${VERSION}" $R4
147:   ${If} $R4 == 0
148:     IfSilent omb_existing_install_current_silent
149:     MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder $OMB_ExistingInstallVersion is already
installed.$\r$\n\r$\nThis installer is also version ${VERSION}, so setup will stop without changing the app."
150:     Quit
151:
152:     omb_existing_install_current_silent:
153:     DetailPrint "OMB Portfolio Builder $OMB_ExistingInstallVersion is already installed. Setup stopped
because the installed version is current."
154:     Quit
155:   ${ElseIf} $R4 == 1
156:     IfSilent omb_existing_install_newer_silent
157:     MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder $OMB_ExistingInstallVersion is already
installed.$\r$\n\r$\nThis installer is older (${VERSION}), so setup will stop without changing the app."
158:     Quit
159:
160:     omb_existing_install_newer_silent:
161:     DetailPrint "OMB Portfolio Builder $OMB_ExistingInstallVersion is newer than installer ${VERSION}.
Setup stopped."
162:     Quit
163:   ${EndIf}
164: FunctionEnd

```

OMBCheckKnownBuilderWorkspace (build/installer.nsh, lines 166-190)

OMBCheckKnownBuilderWorkspace named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 166-190 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

166: Function OMBCheckKnownBuilderWorkspace
167:   IfFileExists "$PROFILE\OMB\builder\desktop-builder\package.json" 0 +3
168:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n\r$\n$PROFILE\OMB\builder\desktop-builder$\r$\n\r$\nRemove, uninstall, or rename that copy
before installing this release. This prevents an older working builder from staying on the machine beside the
installed app."
169:   Quit
170:
171:   IfFileExists "$DOCUMENTS\OMB\desktop-builder\package.json" 0 +3
172:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n\r$\n$DOCUMENTS\OMB\desktop-builder$\r$\n\r$\nRemove, uninstall, or rename that copy before
installing this release. This prevents an older working builder from staying on the machine beside the installed
app."
173:   Quit
174:
175:   IfFileExists "$PROFILE\OneDrive\Documents\OMB\desktop-builder\package.json" 0 +3
176:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n\r$\n$PROFILE\OneDrive\Documents\OMB\desktop-builder$\r$\n\r$\nRemove, uninstall, or rename
that copy before installing this release. This prevents an older working builder from staying on the machine
beside the installed app."
177:   Quit
178:
179:   IfFileExists "$PROFILE\Documents\OMB\desktop-builder\package.json" 0 +3
180:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n\r$\n$PROFILE\Documents\OMB\desktop-builder$\r$\n\r$\nRemove, uninstall, or rename that copy
before installing this release. This prevents an older working builder from staying on the machine beside the
installed app."
181:   Quit
182:
183:   IfFileExists "$DOCUMENTS\omb-portfolio-builder\package.json" 0 +3
184:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development
copy:$\r$\n\r$\n$DOCUMENTS\omb-portfolio-builder$\r$\n\r$\nRemove, uninstall, or rename that copy before
installing this release."
185:   Quit
186:
187:   IfFileExists "$PROFILE\OneDrive\Documents\omb-portfolio-builder\package.json" 0 +3
188:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development
copy:$\r$\n\r$\n$PROFILE\OneDrive\Documents\omb-portfolio-builder$\r$\n\r$\nRemove, uninstall, or rename that
copy before installing this release."
189:   Quit
190: FunctionEnd

```

OMBWriteDuplicateScanScript (build/installer.nsh, lines 192-272)

OMBWriteDuplicateScanScript named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 192-272 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Hit, IsNullOrEmpty, Contains, Add, DesktopBuilder, not, LiteralPath, BuilderWorkspace, SkipDirectory, Windows, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 198: FileWrite \$0 " if ([string]::IsNullOrEmpty(\$Candidate)) { return }\$r\$`n"; line 200: FileWrite \$0 " if (\$\$resolved -match '\\\\OMB-Portfolio-Builder-Setup-[^\`\\\\]+\\.exe\$`) { return }\$r\$`n"; line 205: FileWrite \$0 " if (-not (Test-Path -LiteralPath (Join-Path \$\$Directory 'package.json'))) { return \$\$false }\$r\$`n"; line 206: FileWrite \$0 " if (-not (Test-Path -LiteralPath (Join-Path \$\$Directory 'main.cjs'))) { return \$\$false }\$r\$`n". There are 8 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
192: Function OMBWriteDuplicateScanScript
193:     InitPluginsDir
194:     FileOpen $0 "$PLUGINSIDIR\omb-duplicate-scan.ps1" w
195:     FileWrite $0 "$$ErrorActionPreference = 'SilentlyContinue'$r$`n"
196:     FileWrite $0 "$$hits = New-Object System.Collections.Generic.List[string]$r$`n"
197:     FileWrite $0 "function Add-Hit([string]$$Kind, [string]$$Candidate) {$r$`n"
198:     FileWrite $0 "    if ([string]::IsNullOrEmpty($Candidate)) { return }$r$`n"
199:     FileWrite $0 "    try { $$resolved = (Resolve-Path -LiteralPath $$Candidate -ErrorAction Stop).Path }
catch { $$resolved = $$Candidate }$r$`n"
200:     FileWrite $0 "    if ($$resolved -match '\\\\OMB-Portfolio-Builder-Setup-[^\`\\\\]+\\.exe$`) { return
}$r$`n"
201:     FileWrite $0 "    $$line = $$Kind + ' - ' + $$resolved$r$`n"
202:     FileWrite $0 "    if (-not $$hits.Contains($$line)) { [void]$$hits.Add($$line) }$r$`n"
203:     FileWrite $0 "}$r$`n"
204:     FileWrite $0 "function Test-DesktopBuilder([string]$$Directory) {$r$`n"
205:     FileWrite $0 "    if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'package.json'))) { return
$$false }$r$`n"
206:     FileWrite $0 "    if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'main.cjs'))) { return $$false
}$r$`n"
207:     FileWrite $0 "    if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'builder-site'))) { return
$$false }$r$`n"
208:     FileWrite $0 "    try { $$packageText = Get-Content -LiteralPath (Join-Path $$Directory 'package.json')
-Raw -ErrorAction Stop } catch { return $$false }$r$`n"
209:     FileWrite $0 "    return $$packageText -match 'OMB Portfolio Builder|omb-portfolio-
builder|portfoliobuilder'$r$`n"
210:     FileWrite $0 "}$r$`n"
211:     FileWrite $0 "function Test-BuilderWorkspace([string]$$Directory) {$r$`n"
212:     FileWrite $0 "    if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'package.json'))) { return
$$false }$r$`n"
213:     FileWrite $0 "    if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'main.cjs'))) { return $$false
}$r$`n"
214:     FileWrite $0 "    if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'server.mjs'))) { return
$$false }$r$`n"
215:     FileWrite $0 "    try { $$packageText = Get-Content -LiteralPath (Join-Path $$Directory 'package.json')
-Raw -ErrorAction Stop } catch { return $$false }$r$`n"
216:     FileWrite $0 "    return $$packageText -match 'OMB Portfolio Builder|omb-portfolio-
builder|portfoliobuilder'$r$`n"
217:     FileWrite $0 "}$r$`n"
218:     FileWrite $0 "function Should-SkipDirectory([string]$$Directory) {$r$`n"
219:     FileWrite $0 "    $$name = Split-Path -Leaf $$Directory$r$`n"
220:     FileWrite $0 "    $$managedRoot = Join-Path $Env:LOCALAPPDATA 'OMB Portfolio Builder'$r$`n"
221:     FileWrite $0 "    $$legacyRoot = Join-Path $Env:USERPROFILE 'OMB'$r$`n"
222:     FileWrite $0 "    $$managedBuilder = Join-Path $$managedRoot 'builder'$r$`n"
223:     FileWrite $0 "    $$managedPortfolio = Join-Path $$managedRoot 'portfolio'$r$`n"
224:     FileWrite $0 "    $$legacyBuilder = Join-Path $$legacyRoot 'builder'$r$`n"
225:     FileWrite $0 "    $$legacyPortfolio = Join-Path $$legacyRoot 'portfolio'$r$`n"
226:     FileWrite $0 "    if ($$Directory -ieq $$managedBuilder -or $$Directory -ieq $$managedPortfolio -or
$$Directory -ieq $$legacyBuilder -or $$Directory -ieq $$legacyPortfolio) { return $$true }$r$`n"
227:     FileWrite $0 "    if ($$name -in @('node_modules', '.git', 'dist', '.vs', '$$Recycle.Bin', 'System Volume
Information', 'WinSxS')) { return $$true }$r$`n"
228:     FileWrite $0 "    if ($$Directory -match '\\\\Windows\\\\($$)') { return $$true }$r$`n"
229:     FileWrite $0 "    return $$false$r$`n"
230:     FileWrite $0 "}$r$`n"
231:     FileWrite $0 "function Scan-Tree([string]$$Root) {$r$`n"
232:     FileWrite $0 "    if (-not (Test-Path -LiteralPath $$Root)) { return }$r$`n"
233:     FileWrite $0 "    $$stack = New-Object System.Collections.Generic.Stack[string]$r$`n"
234:     FileWrite $0 "    $$stack.Push($$Root)$r$`n"
235:     FileWrite $0 "    while ($$stack.Count -gt 0) {$r$`n"
236:     FileWrite $0 "        $$dir = $$stack.Pop()$r$`n"
237:     FileWrite $0 "        if (Should-SkipDirectory $$dir) { continue }$r$`n"
238:     FileWrite $0 "        if ((Split-Path -Leaf $$dir) -ieq 'desktop-builder' -and (Test-DesktopBuilder $$dir))
{ Add-Hit 'Local desktop-builder workspace' $$dir }$r$`n"
239:     FileWrite $0 "        if ((Split-Path -Leaf $$dir) -ieq 'builder' -and (Test-BuilderWorkspace $$dir)) {
Add-Hit 'Local builder workspace' $$dir }$r$`n"
240:     FileWrite $0 "        try { $$children = Get-ChildItem -LiteralPath $$dir -Force -ErrorAction Stop } catch
{ continue }$r$`n"
241:     FileWrite $0 "        foreach ($$child in $$children) {$r$`n"
```

```

242: FileWrite $0 "      if ($$child.PSIsContainer) {$r$`n"
243: FileWrite $0 "      if (-not (Should-SkipDirectory $$child.FullName)) {
$$stack.Push($$child.FullName) }$r$`n"
244: FileWrite $0 "      } else {$r$`n"
245: FileWrite $0 "      if ($$child.Name -ieq 'OMB Portfolio Builder.exe') { Add-Hit 'Installed app
executable' $$child.FullName }$r$`n"
246: FileWrite $0 "      if ($$child.Name -like 'OMB-Portfolio-Builder-Portable-*.exe') { Add-Hit
'Portable app executable' $$child.FullName }$r$`n"
247: FileWrite $0 "      }$r$`n"
248: FileWrite $0 "      }$r$`n"
249: FileWrite $0 "      }$r$`n"
250: FileWrite $0 "    }$r$`n"
251: FileWrite $0 "    $$registryKeys = @('HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*', 'HKCU:\S
oftware\Microsoft\Windows\CurrentVersion\Uninstall\*')$r$`n"
252: FileWrite $0 "    foreach ($$key in $$registryKeys) {$r$`n"
253: FileWrite $0 "      Get-ItemProperty $$key -ErrorAction SilentlyContinue | Where-Object { $_.DisplayName
-like 'OMB Portfolio Builder*' } | ForEach-Object {$r$`n"
254: FileWrite $0 "        if ($_.InstallLocation) { Add-Hit 'Registered Windows installation'
$_InstallLocation }$r$`n"
255: FileWrite $0 "        elseif ($_.DisplayIcon) { Add-Hit 'Registered Windows installation' $_DisplayIcon
}$r$`n"
256: FileWrite $0 "      }$r$`n"
257: FileWrite $0 "    }$r$`n"
258: FileWrite $0 "    $$roots = New-Object System.Collections.Generic.List[string]$r$`n"
259: FileWrite $0 "    Get-CimInstance Win32_LogicalDisk -Filter 'DriveType=3' -ErrorAction SilentlyContinue |
ForEach-Object { [void]$$roots.Add($_.DeviceID + '\')} $r$`n"
260: FileWrite $0 "    $$programFilesX86 = [Environment]::GetEnvironmentVariable('ProgramFiles(x86)')$r$`n"
261: FileWrite $0 "    foreach ($$common in @($$env:ProgramFiles, $$programFilesX86, (Join-Path
$$env:LOCALAPPDATA 'Programs'), $$env:USERPROFILE, (Join-Path $$env:PUBLIC 'Desktop')) { if ($$common -and -not
$$roots.Contains($$common)) { [void]$$roots.Add($$common) } }$r$`n"
262: FileWrite $0 "    foreach ($$root in $$roots) { Scan-Tree $$root }$r$`n"
263: FileWrite $0 "    $$unique = $$hits | Sort-Object -Unique$r$`n"
264: FileWrite $0 "    if ($$unique.Count -gt 0) {$r$`n"
265: FileWrite $0 "      $$preview = $$unique | Select-Object -First 10$r$`n"
266: FileWrite $0 "      if ($$unique.Count -gt 10) { $$preview += ('... and ' + ($$unique.Count - 10) + '
more')} $r$`n"
267: FileWrite $0 "      Write-Output ($$preview -join [Environment]::NewLine)$r$`n"
268: FileWrite $0 "      exit 42$r$`n"
269: FileWrite $0 "    }$r$`n"
270: FileWrite $0 "    exit 0$r$`n"
271: FileClose $0
272: FunctionEnd

```

OMBScanForDuplicateBuilderCopies (build/installer.nsh, lines 274-288)

This function runs the duplicate-copy scanner generated by the installer script and blocks setup when another builder copy is found.

Duplicate installations were causing shortcuts and updates to point to the wrong app. This function is part of the system-wide guard against that.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

274: Function OMBScanForDuplicateBuilderCopies
275:   DetailPrint "Scanning this machine for existing OMB Portfolio Builder copies."
276:   Call OMBWriteDuplicateScanScript
277:   nsExec:ExecToStack 'powershell.exe -NoProfile -ExecutionPolicy Bypass -File "$PLUGINS\DIR\omb-
duplicate-scan.ps1"
278:   Pop $R4
279:   Pop $R3
280:
281:   ${If} $R4 == 42
282:     MessageBox MB_OK|MB_ICONSTOP "Setup found another OMB Portfolio Builder copy on this
machine.$r$`n$r$`n$R3$r$`n$r$`nTo prevent duplicate installations and stale builder features, remove,
uninstall, or rename the existing copy first, then run this installer again."
283:     Quit
284:     ${ElseIf} $R4 != 0
285:       MessageBox MB_OK|MB_ICONSTOP "Setup could not complete the duplicate-installation scan. To avoid
duplicate builder copies, setup will stop. Run the installer again as Administrator, or remove old OMB Portfolio
Builder copies manually before installing."
286:       Quit
287:     ${EndIf}
288: FunctionEnd

```

OMBStopBuilderProcessesForUpdate (build/installer.nsh, lines 290-296)

This function stops running builder processes before setup tries to replace files.

Windows cannot update files that are still locked by a running application. This makes the update path much smoother.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
290: Function OMBStopBuilderProcessesForUpdate
291:   DetailPrint "Stopping running OMB Portfolio Builder processes before update."
292:   nsExec::ExecToStack 'cmd /c taskkill /IM "OMB Portfolio Builder.exe" /F /T'
293:   Pop $R4
294:   Pop $R3
295:   Sleep 1500
296: FunctionEnd
```

OMBDisableBuiltInOldUninstallerForInPlaceUpdate (build/installer.nsh, lines 298-313)

OMBDisableBuiltInOldUninstallerForInPlaceUpdate named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 298-313 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
298: Function OMBDisableBuiltInOldUninstallerForInPlaceUpdate
299:   DetailPrint "Preparing in-place update. The old uninstaller handoff will be skipped."
300:
301:   SetRegView 64
302:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
303:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
304:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
305:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
306:
307:   SetRegView 32
308:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
309:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
310:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
311:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
312:   SetRegView lastused
313: FunctionEnd
```

OMBUninstallExistingInstallIfPresent (build/installer.nsh, lines 315-497)

This installer function checks for existing installations and decides whether setup is a fresh install, blocked duplicate install, or in-place update.

It protects the machine from multiple conflicting builder installs and allows update installs to reuse the correct location.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
315: Function OMBUninstallExistingInstallIfPresent
316:   StrCpy $OMB_ExistingInstallLocation ""
317:   StrCpy $OMB_ExistingInstallVersion ""
318:   StrCpy $OMB_IsUpdateInstall "0"
319:   StrCpy $OMB_ManualApplicationInstall "0"
320:   StrCpy $R7 ""
321:   StrCpy $R6 ""
```

```

322: StrCpy $R5 ""
323:
324: SetRegView 64
325: ReadRegStr $R7 HKCU "Software\${APP_GUID}" InstallLocation
326: ReadRegStr $R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
327: ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
328: ${If} $R5 == ""
329: ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
330: ${EndIf}
331: ${If} $R7 == ""
332: ReadRegStr $R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
333: ${EndIf}
334:
335: ${If} $R7 == ""
336: ReadRegStr $R7 HKLM "Software\${APP_GUID}" InstallLocation
337: ReadRegStr $R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
338: ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
339: ${If} $R5 == ""
340: ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
341: ${EndIf}
342: ${If} $R7 == ""
343: ReadRegStr $R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
344: ${EndIf}
345: ${EndIf}
346:
347: ${If} $R7 == ""
348: SetRegView 32
349: ReadRegStr $R7 HKCU "Software\${APP_GUID}" InstallLocation
350: ReadRegStr $R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
351: ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
352: ${If} $R5 == ""
353: ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
354: ${EndIf}
355: ${If} $R7 == ""
356: ReadRegStr $R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
357: ${EndIf}
358: ${If} $R7 == ""
359: ReadRegStr $R7 HKLM "Software\${APP_GUID}" InstallLocation
360: ReadRegStr $R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
361: ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
362: ${If} $R5 == ""
363: ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
364: ${EndIf}
365: ${If} $R7 == ""
366: ReadRegStr $R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
367: ${EndIf}
368: ${EndIf}
369: SetRegView lastused
370: ${EndIf}
371:
372: ${If} $R7 == ""
373: IfFileExists "$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
374: StrCpy $R7 "$LOCALAPPDATA\Programs\OMB Portfolio Builder"
375: StrCpy $R5 "$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
376: ${EndIf}
377: ${If} $R7 == ""
378: IfFileExists "$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
379: StrCpy $R7 "$PROGRAMFILES64\OMB Portfolio Builder"
380: StrCpy $R5 "$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
381: ${EndIf}
382: ${If} $R7 == ""
383: IfFileExists "$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
384: StrCpy $R7 "$PROGRAMFILES\OMB Portfolio Builder"
385: StrCpy $R5 "$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
386: ${EndIf}
387: ${If} $R7 == ""
388: IfFileExists "$PROFILE\OMB\application\OMB Portfolio Builder\OMB Portfolio Builder.exe" 0 +4
389: StrCpy $R7 "$PROFILE\OMB\application\OMB Portfolio Builder"
390: StrCpy $R6 "a legacy app-folder copy"
391: StrCpy $OMB_ManualApplicationInstall "1"
392: ${EndIf}
393: ${If} $R7 == ""
394: IfFileExists "$PROFILE\OMB\application\OMB Portfolio Builder.exe" 0 +4
395: StrCpy $R7 "$PROFILE\OMB\application"
396: StrCpy $R6 "a standalone app-folder copy"
397: StrCpy $OMB_ManualApplicationInstall "1"

```

```

398:     ${EndIf}
399:     ${If} $R7 == ""
400:         IfFileExists "$PROFILE\OneDrive\Documents\OMB\application\OMB Portfolio Builder\OMB Portfolio
Builder.exe" 0 +4
401:         StrCpy $R7 "$PROFILE\OneDrive\Documents\OMB\application\OMB Portfolio Builder"
402:         StrCpy $R6 "a legacy app-folder copy"
403:         StrCpy $OMB_ManualApplicationInstall "1"
404:     ${EndIf}
405:     ${If} $R7 == ""
406:         IfFileExists "$PROFILE\OneDrive\Documents\OMB\application\OMB Portfolio Builder.exe" 0 +4
407:         StrCpy $R7 "$PROFILE\OneDrive\Documents\OMB\application"
408:         StrCpy $R6 "a standalone app-folder copy"
409:         StrCpy $OMB_ManualApplicationInstall "1"
410:     ${EndIf}
411:     ${If} $R7 == ""
412:         IfFileExists "$PROFILE\OneDrive\Desktop\OMB\application\OMB Portfolio Builder.exe" 0 +4
413:         StrCpy $R7 "$PROFILE\OneDrive\Desktop\OMB\application"
414:         StrCpy $R6 "a standalone app-folder copy"
415:         StrCpy $OMB_ManualApplicationInstall "1"
416:     ${EndIf}
417:     ${If} $R7 == ""
418:         IfFileExists "$DOCUMENTS\OMB\application\OMB Portfolio Builder.exe" 0 +4
419:         StrCpy $R7 "$DOCUMENTS\OMB\application"
420:         StrCpy $R6 "a standalone app-folder copy"
421:         StrCpy $OMB_ManualApplicationInstall "1"
422:     ${EndIf}
423:
424:     ${If} $OMB_ManualApplicationInstall == "1"
425:         StrCpy $R5 ""
426:     ${EndIf}
427:
428:     ${If} $R7 != ""
429:         ${If} $OMB_ManualApplicationInstall == "1"
430:             Call OMBReadExistingAppVersion
431:         ${Else}
432:             StrCpy $OMB_ExistingInstallVersion $R6
433:         ${EndIf}
434:         ${If} $R7 == "$PROFILE\OMB\application\OMB Portfolio Builder"
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

OMBUninstallPageCreate (build/installer.nsh, lines 546-560)

OMBUninstallPageCreate named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 546-560 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

546: Function OMBUninstallPageCreate
547:     nsDialogs::Create 1018
548:     Pop $OMB_UninstallDialog
549:     ${If} $OMB_UninstallDialog == error
550:         Abort
551:     ${EndIf}
552:
553:     ${NSD_CreateLabel} 0 0 100% 12u "Uninstall OMB Portfolio Builder"
554:     Pop $0
555:
556:     ${NSD_CreateLabel} 0 20u 100% 90u "Uninstall will remove:$\r$\n$\r$\n- OMB Portfolio Builder
application files$\r$\n- OMB Portfolio Builder shortcuts$\r$\n- Local app data created by this application when
Windows requests app-data cleanup$\r$\n$\r$\nUninstall will not silently remove shared tools such as Git for
Windows, Node.js, pnpm, browsers, or your Git repositories."
557:     Pop $0
558:
559:     nsDialogs::Show
560: FunctionEnd

```

Representative opening snippet

```

1: !include LogicLib.nsh
2: !include nsDialogs.nsh
3: !include WinMessages.nsh
4: !include WordFunc.nsh
5:
6: !ifndef BUILD_UNINSTALLER
7: Var OMB_ToolsDialog
8: Var OMB_ShortcutCheckbox

```

```

9: Var OMB_PublishingToolsCheckbox
10: Var OMB_CreateDesktopShortcutState
11: Var OMB_InstallPublishingToolsState
12: Var OMB_ExistingInstallLocation
13: Var OMB_ExistingInstallVersion
14: Var OMB_IsUpdateInstall
15: Var OMB_ManualApplicationInstall
16:
17: Function OMBToolsPageCreate
18:   nsDialogs::Create 1018
19:   Pop $OMB_ToolsDialog
20:   ${If} $OMB_ToolsDialog == error
21:     Abort
22:   ${EndIf}
23:
24:   ${NSD_CreateLabel} 0 0 100% 12u "Tools and shortcuts"
25:   Pop $0
26:
27:   ${NSD_CreateLabel} 0 18u 100% 36u "The desktop app includes its own runtime. Users do not need to
install Node.js or npm.$\r$\nGit for Windows with Git Credential Manager is only needed when publishing to
GitHub."
28:   Pop $0
29:
30:   ${NSD_CreateCheckbox} 0 62u 100% 12u "Create a desktop shortcut"
31:   Pop $OMB_ShortcutCheckbox
32:   ${If} $OMB_CreateDesktopShortcutState == ""
33:     StrCpy $OMB_CreateDesktopShortcutState ${BST_CHECKED}
34:   ${EndIf}
35:   ${If} $OMB_CreateDesktopShortcutState == ${BST_CHECKED}
36:     ${NSD_Check} $OMB_ShortcutCheckbox
37:   ${EndIf}
38:
39:   ${NSD_CreateCheckbox} 0 82u 100% 24u "Check publishing tools and install Git for Windows if Git/Git
Credential Manager is missing"
40:   Pop $OMB_PublishingToolsCheckbox
41:   ${If} $OMB_InstallPublishingToolsState == ""
42:     StrCpy $OMB_InstallPublishingToolsState ${BST_CHECKED}
43:   ${EndIf}
44:   ${If} $OMB_InstallPublishingToolsState == ${BST_CHECKED}
45:     ${NSD_Check} $OMB_PublishingToolsCheckbox
46:   ${EndIf}
47:
48:   ${NSD_CreateLabel} 0 114u 100% 38u "Existing shared tools are not uninstalled silently. If a compatible
tool is already available, setup skips it. If Git is missing or unusable, setup will try to install Git for
Windows side-by-side using Windows Package Manager."
49:   Pop $0
50:
51:   nsDialogs::Show
52: FunctionEnd
53:
54: Function OMBToolsPageLeave
55:   ${NSD_GetState} $OMB_ShortcutCheckbox $OMB_CreateDesktopShortcutState
56:   ${NSD_GetState} $OMB_PublishingToolsCheckbox $OMB_InstallPublishingToolsState
57:   ${If} $OMB_IsUpdateInstall == "1"
58:     ${AndIf} $INSTDIR != $OMB_ExistingInstallLocation
59:     MessageBox MB_OK|MB_ICONSTOP "OMB Portfolio Builder is already installed on this
machine.$\r$\n$\r$\nUpdates must use the current installed
location:$\r$\n$OMB_ExistingInstallLocation$\r$\n$\r$\nSetup will not create another installed copy in a
different directory."
60:     Abort
61:   ${EndIf}
62: FunctionEnd
63:
64: Function OMBCheckGitAvailable
65:   StrCpy $R9 "0"
66:   IfFileExists "$PROGRAMFILES64\Git\cmd\git.exe" 0 +2
67:   StrCpy $R9 "1"
68:   IfFileExists "$PROGRAMFILES\Git\cmd\git.exe" 0 +2
69:   StrCpy $R9 "1"
70:   IfFileExists "$PROGRAMFILES32\Git\cmd\git.exe" 0 +2
71:   StrCpy $R9 "1"
72:   IfFileExists "$LOCALAPPDATA\Programs\Git\cmd\git.exe" 0 +2

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.